Mzansi Builds: Documentation

Tarika Sukdeoa
2026

Mzanzi Builds: Documentation **1**
How to run: 3
Technology used: 3
The following workflows were used: 3
Use of AI: 3
Backend 4
Design Principles used: 4
Description of files 5
Frontend: React app 11

How to run:

Open in VS code
Open one cmd terminal:

- type: cd backend
- type: npm run dev

Open another cmd terminal

- type: cd client
- type: npm start

Technology used:

This web application uses the MERN stack and utilises the MVC design pattern.

The following workflows were used:

- Used Express for the backend development and api production.
- Used MongoDB for database management.
- Used React for the frontend development
- Used postman for testing backend
- Used coolors to find the colour palette.

Use of AI:

Deepseek was used for the following:

- to create some of the tables that are in this document
- used for the graphics and layout of the web pages.
- To aid in debugging, example identifying syntax errors or parentheses that may have been missed

Backend

Design Principles used:

File	Principle	Description
database.js	SRP	Only handles the interaction with the database
database.js	OCP	Can be extended but cannot be altered
users.js	Liskov	Can be swapped out with other models
user.js	Interface segregation	Only provides necessary user methods
authService.js	SRP	Solely responsible for authentication
authService.js	DIP	Only depends on abstract structures, not concrete ones.
authController.js	SRP	
AppError.js	SRP, Open closed principle	
authRoutes.js	SRP	
projectrepository.js	SRP, Repository pattern	
projectService.js	SRP, DIP	

\

Description of files

File	Description
user.js	Handles the user's data for the sign-up and login. Uses the bcrypt package to hash passwords for security. Records timestamps for the user data. Handles logic for password verification. Holds passwords back from JSON file conversion.
project.js	Handles the project management.
authService.js	Handles the authentication methods. Has methods for generating tokens, registering users, logging in users, and finding users through their userid. Handles user authentication on the mongoDB side.
authController	Handles the user authentication on the server side managing the requests and the responses. Calls the appropriate methods from the authService class.
AppError.js	Manages the error handling
authRotes.js	Manages the routes to the server such as login, sign up and dash board.
auth.js	Handles the token authorization. Restriucts who is able to execute certain actions and carry out authentication.
server.js	Manages connection to the server. Orchestrates the routes with the app interface. Listens to the port.
projectService.js	This is the business layer that interacts with the project repository. It contains all of the validation and error handling whilst calling the CRUD methods from the projectRespository class.
projectRoute.js	This acts as an interface between the projectController, the express app and the authMiddleware. Refer to table 3.
projectRepository.js	Handles all the CRUD operations for the Project model. Please refer to table 2 for a breakdown of the various CRUD methods
projectController.js	Manages the Reqests and responses to and from the database for the CRUD operations on the project model. Uses the CRUD methods from the projectService

	class.
collaborationService.js	Contains the business logic for the collaboration requests. Interacts with the database via the collaborationRequestRepository
CollaborationRequest.js	Creates the schema or model for the collaboration requests using mongoDB
collaborationRequestRepository.js	Handles all the database operations for the collaboration requests.
collaborationController.js	Handles the https requests for the collaboration requests.
collaborationRoutes	This acts as an interface between the CollaborationController, the express app and the authMiddleware.

AppError error codes:

Code	Explination
400	Bad request, missing fields
401	Wrong password
403	Forbidden
404	Not found
409	Conflict, email exists
500	Server error

Table 1: Tests performed on the server:

Table generated using deepseek

✅ Test Summary Table

Test #	Test Name	Endpoint	Method	Input
1	Register New User	`/api/auth/register`	POST	`{username:"testuser", email:"test@example.com", password:"123456"}`
2	Register Duplicate User	`/api/auth/register`	POST	Same as Test 1
3	Register Invalid Email	`/api/auth/register`	POST	`{username:"baduser", email:"not-an-email", password:"123456"}`
4	Register Short Password	`/api/auth/register`	POST	`{username:"baduser2", email:"bad@example.com", password:"123"}`
5	Valid Login	`/api/auth/login`	POST	`{email:"test@example.com", password:"123456"}`
6	Wrong Password Login	`/api/auth/login`	POST	`{email:"test@example.com", password:"wrongpass"}`
7	Non-existent Email Login	`/api/auth/login`	POST	`{email:"nonexistent@example.com", password:"123456"}`
8	Protected Route (Valid Token)	`/api/auth/me`	GET	Header: `Authorization: Bearer [valid_token]`
9	Protected Route (No Token)	`/api/auth/me`	GET	No Authorization header
10	Protected Route (Invalid Token)	`/api/auth/me`	GET	Header: `Authorization: Bearer invalid123`

Table 2: CRUD operations for Project model

Generated using deepseek

Method	Purpose	Returns
`create()`	Create new project	Project
`findAllActive()`	Live feed	Array of projects
`findById()`	Get single project	Project
`findByOwner()`	User's all projects	Array
`findActiveByOwner()`	User's active projects	Array
`findCompletedByOwner()`	User's completed	Array
`addMilestone()`	Add achievement	Updated project
`completeProject()`	Mark completed	Updated project
`addCollaborator()`	Add team member	Updated project
`removeCollaborator()`	Remove team member	Updated project
`getGlobalCompletedProjects()`	Celebration wall	Array (limited)
`updateStage()`	Change project stage	Updated project
`deleteProject()`	Remove project	Deleted project
`searchProjects()`	Search by title/desc	Array
`findByStage()`	Filter by stage	Array
`countUserProjects()`	Project count	Number
`countUserCompletedProjects()`	Completed count	Number

Table 3: Routes for the projectRoutes.js file

Generated using deepseek

Method	Endpoint	Controller Method	Auth Required	Purpose
GET	`/api/projects`	`getAllActiveProjects()`	No	Live feed
GET	`/api/projects/:id`	`getProjectById()`	No	View project
GET	`/api/projects/search`	`searchProjects()`	No	Search
GET	`/api/projects/stage/:stage`	`getProjectsByStage()`	No	Filter by stage
GET	`/api/projects/celebrations`	`getCelebrationWall()`	No	Celebration wall
POST	`/api/projects`	`createProject()`	Yes	Create project
GET	`/api/projects/my-projects`	`getMyProjects()`	Yes	My projects
GET	`/api/projects/my-projects/active`	`getMyActiveProjects()`	Yes	My active
GET	`/api/projects/my-projects/completed`	`getMyCompletedProjects()`	Yes	My completed
GET	`/api/projects/my-stats`	`getUserStats()`	Yes	My stats
POST	`/api/projects/:id/milestones`	`addMilestone()`	Yes	Add milestone
PATCH	`/api/projects/:id/complete`	`completeProject()`	Yes	Complete project
POST	`/api/projects/:id/collaborate`	`requestCollaboration()`	Yes	Request to join
DELETE	`/api/projects/:id/collaborators/:userId`	`removeCollaborator()`	Yes	Remove member
PATCH	`/api/projects/:id/stage`	`updateProjectStage()`	Yes	Update stage
DELETE	`/api/projects/:id`	`deleteProject()`	Yes	Delete project

Table 4: Test project routes

#	Method	Route	Controller Method	Purpose
1	POST	`/api/auth/register`	`register`	Create new user account
2	POST	`/api/auth/login`	`login`	Login existing user
3	GET	`/api/auth/me`	`getMe`	Get current user profile
4	GET	`/api/projects/`	`getAllActiveProjects`	Live feed - view all active projects
5	GET	`/api/projects/:id`	`getProjectById`	View single project details
6	POST	`/api/projects/`	`createProject`	Create a new project
7	GET	`/api/projects/my/projects`	`getMyProjects`	Get all projects owned by current user
8	GET	`/api/projects/my/stats`	`getUserStats`	Get user's project statistics
9	POST	`/api/projects/:id/edit/milestones`	`addMilestone`	Add milestone to project
10	PATCH	`/api/projects/:id/edit/complete`	`completeProject`	Mark project as completed

Frontend: React app

File	description
Register.jsx	Handles validation for the registration page. Returns an html code for the registration page.
Dashboard.jsx	Handles validation for and returns the html file for the dashboard page.
CreateProject.jsx	Handles validation for and returns the html file for the create project page.
Celebrationwall.jsx	Displays the 20 projects that have most recently been completed.
App.js	Manages all the routes between web pages.
api.js	Handles the interactions between the backend and the frontend.
ProjectDetails.js	Responsible for displaying a summary of the project and allows the user to edit the project, add milestones and make amd accept collaboration requests.